

Part 1: Hardware Performance Benchmarking

Activity 1.1: Empirical CPU, Memory, and Storage Benchmarking

The purpose of this activity was to measure the actual performance and resource usage of my computer using Python. The benchmark tested three areas: CPU execution time, memory utilization during a large allocation, and storage read/write throughput.

The tests were performed on my **HP EliteBook 830 G5 running Windows 11 Pro 64-bit**.

```
lab2 python benchmark.py
CPU benchmark: 2.87615 s for 10 runs
Memory usage - before: 88.1%, during: 91.7%, after: 86.7%
Memory - before: 6.92 GB, during: 7.20 GB, after: 6.80 GB
Storage - write: 395.2 MB/s, read: 1156.0 MB/s
BLAISE lab2 main ?4 ~1
```

Small Discussion

The CPU benchmark completed ten runs of the selected CPU-intensive task in **2.87615 seconds**. This provides a baseline measurement of the machine's performance for this particular Python workload.

Memory utilization increased from **88.1% before the allocation to 91.7% during the allocation**, showing that the large data structure increased overall system memory usage. After the allocated data was released, memory utilization decreased to **86.7%**, which was close to the original level.

The storage benchmark measured a write speed of **395.2 MB/s** and a read speed of **1156.3 MB/s**. The read speed was substantially higher than the write speed during this test. These measurements represent the performance observed under this particular test rather than the maximum advertised performance of the storage device.

Overall, the experiment established empirical baseline measurements for the CPU, memory, and storage components of the computer. These measurements provide a useful reference for the benchmarking and monitoring activities that follow.

Activity 1.2: CPU Benchmark Cross-Validation

Machine: HP EliteBook 830 G5
Operating System: Windows 11 Pro 64-bit
Geekbench version: 6.7.1

The CPU was benchmarked using **Geekbench 6.7.1 for Windows AVX2** on the same HP EliteBook 830 G5 used for the Python benchmark.

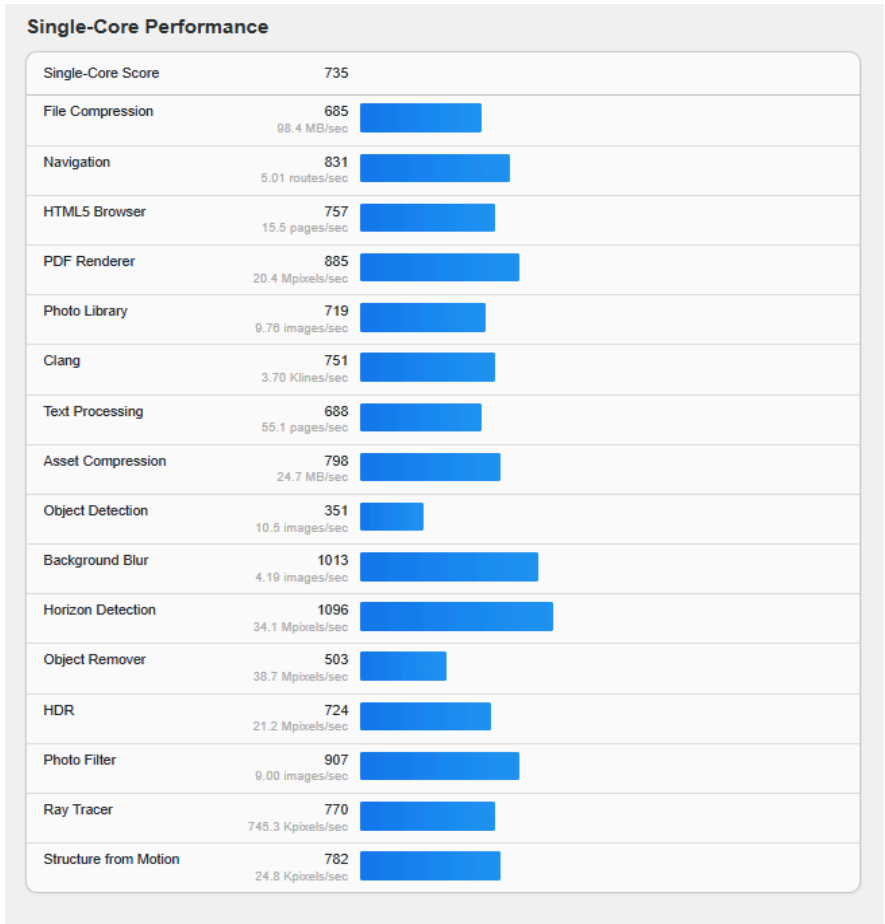
The Python CPU benchmark from Activity 1.1 took **3.39537 seconds for 10 runs**.

The Geekbench benchmark produced the following overall results:

Benchmark	Result
Python CPU benchmark	3.39537 s / 10 runs

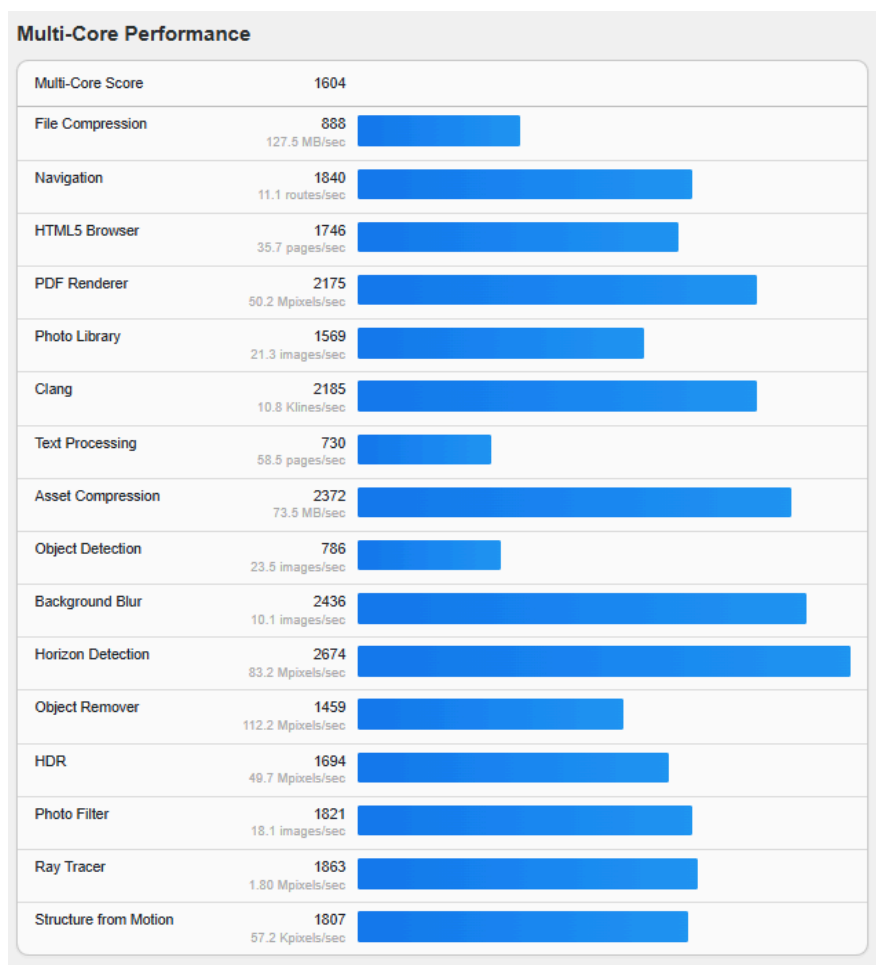
Geekbench Single-Core	735
Geekbench Multi-Core	1604

Geekbench Single-Core Results



The single-core benchmark produced an overall score of **735**. The individual workloads produced different scores, with the highest scores coming from Horizon Detection (**1096**) and Background Blur (**1013**), while Object Detection recorded the lowest score (**351**).

Geekbench Multi-Core Results



The multi-core benchmark produced an overall score of **1604**. The individual workloads varied considerably; Horizon Detection achieved a score of **2674**, while Text Processing recorded **730**.

Comparison with Geekbench Browser

The Geekbench Browser contains multiple submitted Geekbench 6 results for the Intel Core i5-7300U. The results vary between systems. For example, one Windows AVX2 result recorded **1164 single-core and 2114 multi-core**, while another Windows 11 result recorded **1121 single-core and 2218 multi-core**.

This variation shows that benchmark results can depend on factors such as the operating system, power settings, cooling, memory configuration, and the workload being performed. Therefore, the result obtained from my own machine is more directly representative of its condition during this particular test.

Small Discussion

The Python CPU benchmark and Geekbench measure CPU performance using different workloads and scoring systems, so their numerical results cannot be directly compared. The Python benchmark measures the time required for the computer to perform a specific CPU-bound Python operation, whereas Geekbench uses a collection of standardized workloads to produce separate single-core and multi-core scores.

My Python benchmark completed 10 runs in **3.39537 seconds**, while Geekbench produced a **single-core score of 735** and a **multi-core score of 1604**. In general, stronger CPU performance should allow a processor to

complete a comparable CPU-bound workload in less time. However, the relationship is not necessarily proportional because the Python task and Geekbench workloads are different.

The Geekbench Browser results also demonstrate that the same i5-7300U can produce different scores on different systems. For example, published Geekbench 6 results range from around 1100 - 1200 in single-core performance and around 2100–2200 in multi-core performance in the examples examined.

A comparison with a second machine running the same Python benchmark would provide stronger evidence of the relationship between Geekbench scores and the execution time of the Python workload. This comparison was not performed in this experiment.

Activity 1.3: Generational CPU and Performance-per-Watt Comparison

```
lab2 python benchmark.py
CPU benchmark: 2.46870 s for 10 runs
Memory usage - before: 86.2%, during: 89.9%, after: 85.6%
Memory - before: 6.76 GB, during: 7.05 GB, after: 6.72 GB
Storage - write: 370.9 MB/s, read: 1118.1 MB/s

Performance per Watt:
Intel Core i5-7300U: 157.20 points/W
Intel Core i5-8250U: 211.27 points/W
Intel Core i5-10210U: 266.40 points/W
BLAISE lab2 main ?1
```

Three Intel Core i5 mobile processors from different generations were selected for comparison: the 7th-generation Core i5-7300U, the 8th-generation Core i5-8250U, and the 10th-generation Core i5-10210U. Specifications were obtained from Intel's processor documentation and benchmark results were obtained from the Geekbench 6 Browser.

CPU	Generation	Cores / Threads	Base Clock	Boost Clock	TDP	Geekbench 6 Multi-Core	Performance/W
Intel Core i5-7300U	7th	2 / 4	2.60 GHz	3.50 GHz	15 W	2358	157.20
Intel Core i5-8250U	8th	4 / 8	1.60 GHz	3.40 GHz	15 W	3169	211.27
Intel Core i5-10210U	10th	4 / 8	1.60 GHz	4.20 GHz	15 W	3996	266.40

Table 1: Generational CPU comparison

The performance-per-watt values were calculated using:

$$Performance/W = \frac{Geekbench6Multi-CoreScore}{TDP}$$

Architectural discussion

The most significant change between the 7th-generation i5-7300U and the 8th-generation i5-8250U is the increase from **2 cores and 4 threads to 4 cores and 8 threads**, while both processors retain a 15 W TDP. Intel's specifications confirm the 7300U as a 2-core/4-thread processor and the 8250U as a 4-core/8-thread processor.

This increase in core and thread count helps explain the improvement in multi-core performance from **2358 to 3169** in the selected Geekbench 6 results. The performance-per-watt figure consequently increases from **157.20 to 211.27 points/W**, even though both processors have the same 15 W TDP.

The 10th-generation i5-10210U retains the 4-core/8-thread configuration and 15 W TDP but increases maximum turbo frequency to **4.20 GHz**, compared with 3.40 GHz on the i5-8250U. It also has 6 MB of cache compared with 3 MB on the i5-7300U. The selected Geekbench 6 result for the i5-10210U reaches **3996 multi-core**, producing **266.40 points/W**.

The comparison therefore shows that **clock speed and core count alone do not remain constant across generations**. The largest change in this particular comparison occurs between the 7th and 8th generations, where the number of cores and threads doubles without increasing the nominal TDP. Later improvements in the 10th generation include higher turbo frequency and additional architectural refinements, contributing to the higher measured benchmark score. The benchmark results should nevertheless be interpreted as measurements from particular systems rather than as guaranteed performance for every computer using these processors.

Activity 1.4: ARM vs x86 Efficiency

(a) What architectural difference is most responsible for the efficiency gap?

ARM and x86 use different instruction set architectures (ISAs). **ARM is generally designed around a simpler, more regular instruction set and a strong emphasis on power efficiency**, while x86 supports a more complex and extensive instruction set with substantial backward compatibility.

This difference affects how processors decode and execute instructions. ARM designs can place greater emphasis on simple instruction processing and efficient power usage, which is particularly useful in battery-powered devices such as smartphones and tablets. x86 processors, in contrast, have historically prioritized high performance, compatibility with a large existing software ecosystem, and support for complex workloads.

However, the efficiency difference cannot be attributed to the ISA alone. Modern processor performance and power consumption depend on the complete microarchitecture and implementation of the processor.

(b) Is the gap purely architectural, or does the manufacturing process also play a role?

The gap is **not purely architectural**. Manufacturing technology also has an important effect.

A smaller semiconductor process node can allow manufacturers to place more transistors in a given area and can potentially reduce the energy required for switching transistors. This can allow a processor to provide more performance within a particular power budget.

Therefore, comparing an ARM processor manufactured using a newer process with an older x86 processor would not be a fair test of architecture alone. **ISA, microarchitecture, manufacturing process, clock frequency, cache design, core count, workload, and power-management techniques can all affect the observed efficiency.**

This is also relevant to our Activity 1.3 results: the three Intel CPUs we compared were all listed with a **14 nm manufacturing process**, so their performance differences cannot simply be explained by moving to a smaller process node.

(c) Where does x86 retain a practical advantage?

One practical advantage of x86 is **software compatibility**.

A large amount of existing desktop and enterprise software has historically been developed and optimized for x86 systems. Organizations that depend on legacy applications may therefore choose x86 hardware because it can run their existing software without requiring substantial modification or recompilation.

x86 can also be advantageous in environments that depend heavily on established x86 virtualization, operating-system, driver, and enterprise-software ecosystems.

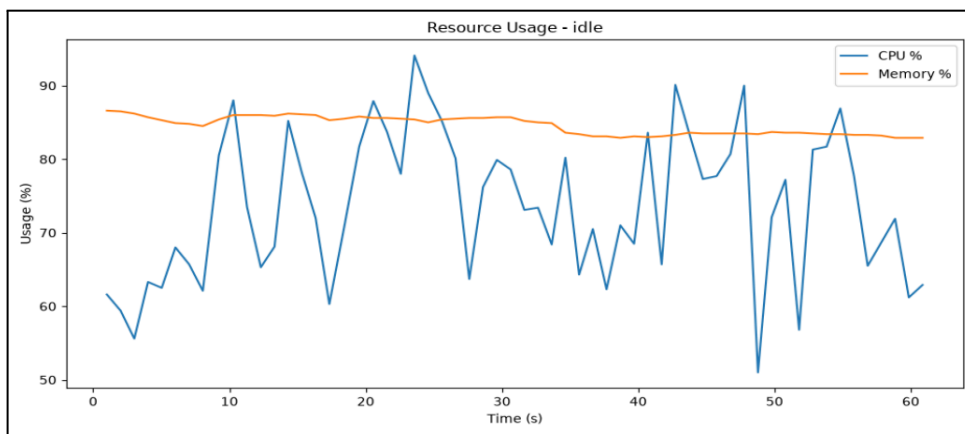
Conclusion

ARM and x86 represent different approaches to processor design, but their efficiency differences cannot be explained by instruction-set architecture alone. Modern efficiency is influenced by microarchitecture, manufacturing technology, power management, workload, and software. ARM has become highly competitive in performance while maintaining strong power efficiency, while x86 continues to have practical advantages in areas such as compatibility with established software ecosystems.

Part 2: Real-Time Monitoring and Dynamic Power Management

Activity 2.1: Real-Time CPU and Memory Monitoring

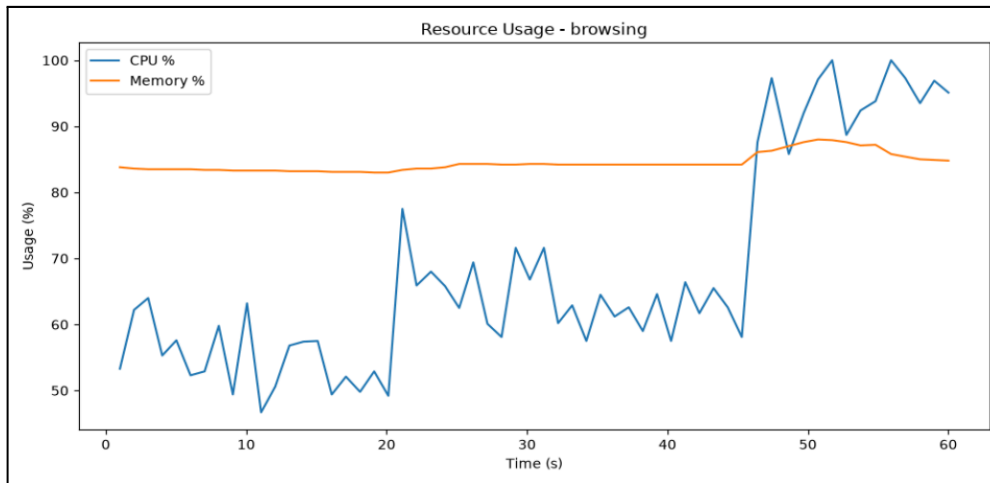
Run 1: Idle



During the supposed idle period, CPU utilization remained relatively high and fluctuated considerably, approximately between **51% and 94%**. Memory utilization was much more stable, remaining around **83–86%** for most of the run.

This indicates that although no deliberate workload was being performed, the computer was still running significant background processes. The high memory utilization also agrees with the high system-wide memory usage observed earlier in Activity 1.1.

Run 2: Web Browsing

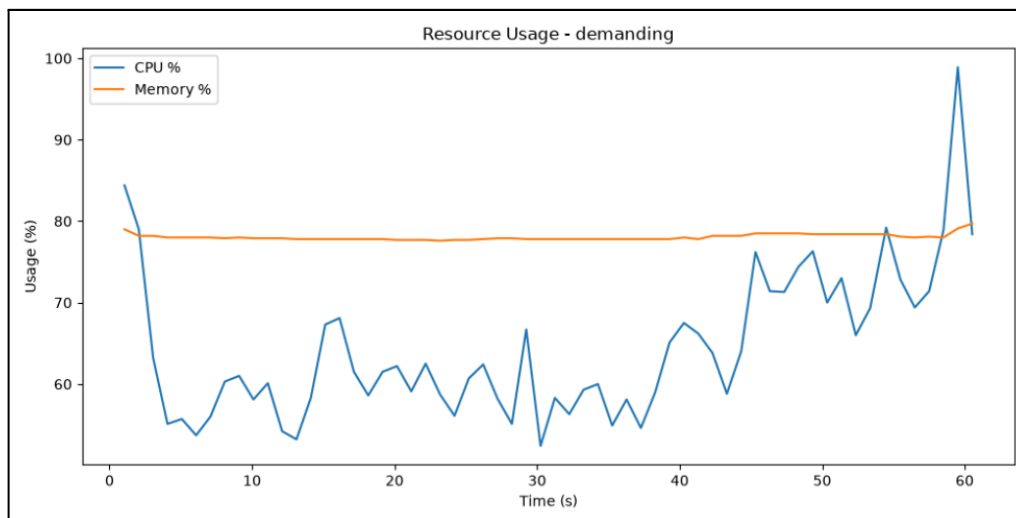


During web browsing, CPU utilization initially fluctuated mostly between approximately **47% and 77%**. Toward the end of the run, CPU utilization increased substantially, reaching approximately **100%**.

Memory utilization remained relatively stable around **83–88%**, with a modest increase during the period of higher CPU activity.

The increase in CPU utilization toward the end suggests that the browsing activity generated additional processing demand, while memory usage changed less dramatically.

Run 3: Demanding Workload



During the demanding workload, CPU utilization generally remained between approximately **53% and 79%**, with a significant peak of approximately **99%** near the end of the test.

Memory utilization was considerably more stable, remaining around **78–79%** throughout most of the experiment.

The CPU therefore showed much greater short-term variation than memory utilization. The sharp CPU peak indicates a period where the demanding task required substantial processing resources.

Overall Comparison

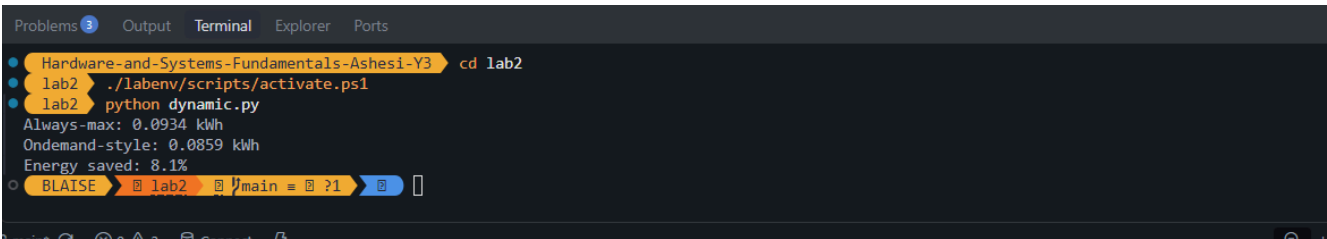
Workload	CPU behavior	Memory behavior
Idle	High and highly variable, ~51–94%	Stable and high, ~83–86%
Browsing	~47–100%, with a strong increase near the end	~83–88%, relatively stable
Demanding	~53–99%, with a major peak near the end	~78–79%, very stable

Small Discussion

The three experiments show that CPU utilization was considerably more variable than memory utilization. The idle run unexpectedly showed relatively high CPU utilization, indicating that background processes were consuming significant processing resources even when no deliberate task was being performed. Web browsing produced a noticeable increase in CPU activity toward the end of the monitoring period. The demanding workload also produced substantial CPU activity and reached almost 100% utilization at its peak.

Memory utilization changed much less than CPU utilization in all three experiments. This is partly because the system already had relatively high memory utilization before the experiments, leaving less visible variation during the short monitoring periods. Overall, the results demonstrate that CPU utilization responds more rapidly to changes in workload, while system-wide memory utilization can remain relatively stable.

Activity 2.3: Dynamic Power Management



```
Hardware-and-Systems-Fundamentals-Ashesi-Y3 cd lab2
lab2 ./labenv/scripts/activate.ps1
lab2 python dynamic.py
Always-max: 0.0934 kWh
Ondemand-style: 0.0859 kWh
Energy saved: 8.1%
BLAISE lab2 main ?1
```

Results

Policy	Energy consumption
Always-max	0.0934 kWh
Ondemand-style	0.0859 kWh
Energy saved	8.1%

The average CPU utilization recorded during this run was approximately **65.2%**.

Under the utilization-proportional model:

$$P(u) = P_{idle} + (P_{max} - P_{idle})u$$

with: $P_{idle} = 73W$ and $P_{max} = 95W$

Small Discussion

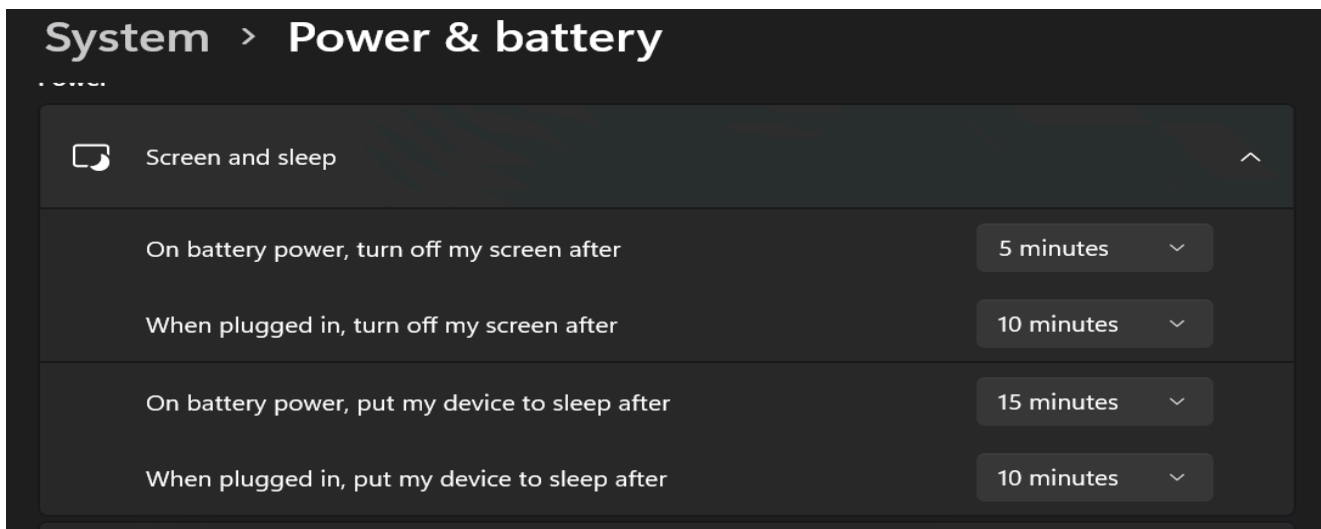
The simulation shows that an ondemand-style power policy would consume approximately 0.0859 kWh during the monitoring window, compared with 0.0934 kWh if the CPU operated continuously at its maximum power of 95 W. This represents an estimated energy saving of approximately 8.1%.

The saving occurs because the ondemand-style model reduces the assumed CPU power when utilization is below 100%, rather than continuously operating at the maximum power level. The result demonstrates how dynamic power management can reduce energy consumption while still allowing higher power to be used when processing demand increases.

Activity 2.4: Practical Power-Saving Techniques

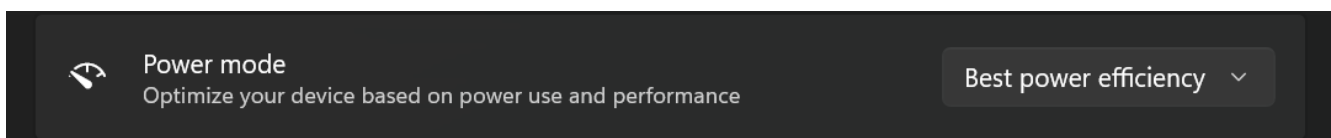
We are required at least **three OS-level power-saving changes**, with before/after measurements where possible. Since we are using Windows 11, we will use these three changes.

Change 1: Reduce the screen timeout



The display timeout was reduced (from 30 minutes to 15 minutes) so that the screen enters a lower-power state sooner when the computer is inactive. This reduces unnecessary display energy consumption during periods when the computer is not being actively used.

Change 2: Select a power-efficient power mode



The Windows power mode was changed to a power-efficient setting. This allows the operating system to favor lower power consumption when possible instead of prioritizing maximum performance continuously.

Change 3: Disable Bluetooth when it is not needed



Bluetooth was disabled because it was not required during the experiment. This prevents the wireless subsystem from remaining unnecessarily active and can therefore reduce power consumption.

Small Discussion

Three OS-level power-saving measures were applied: reducing the display timeout, selecting the Best power efficiency power mode, and disabling Bluetooth when it was not required. These changes target different sources of energy consumption. The display timeout reduces energy used by the screen during inactivity, the power mode allows Windows to favor lower-power operation, and disabling unused Bluetooth prevents unnecessary wireless activity. Exact electrical savings were not measured because the experiment did not use an external power meter.

Activity 2.5: Phantom Load

Let's use a **television** because we have a good official source for standby power.

ENERGY STAR specifies that certified TVs must have standby-passive power of **0.5 W or less**, while network-capable TVs have a standby-active limit of 1.0 W.

For a conservative/simple calculation, we'll use: $P_{standby} = 0.5W$

```
lab2 python phantom_load.py
Monthly phantom load: 0.36 kWh
Monthly phantom load cost: GHC 0.12
BLAISE lab2 main = 2
```

Calculations

Daily energy: $E_{day} = \frac{0.5 \times 24}{1000} E_{day} = 0.012 kWh$

Monthly energy: $E_{month} = 0.012 \times 30 = 0.36 kWh$

So a television consuming 0.5 W continuously in standby would use approximately **0.36 kWh per month**.

GHC cost

Because our estimated monthly consumption is only **0.36 kWh**, it falls into the first tariff bracket: $0.36 \leq 50$

So: $Cost = 0.36 \times 0.34 = 0.1224 \text{ GHC}$ Rounded to two decimal places to give **GHC 0.12/month**

Submission answer

A standby power consumption of 0.5 W was assumed for the television. At 24 hours per day for 30 days, this corresponds to 0.36 kWh per month. Using the ECG tariff function from Lab 1, the estimated monthly electricity cost is approximately **GHC 0.12**.

Although the monthly cost of a single low-power standby device is small, phantom loads can become more significant when several devices remain connected continuously. This demonstrates why reducing unnecessary standby consumption can contribute to overall energy conservation.

Part 3: Predictive Maintenance: S.M.A.R.T. Data

Activity 3.1: Read and Parse S.M.A.R.T. Data

We were required to run “*smartctl -a <device>*” on a drive and use Python's “*subprocess*” and “*re*” modules to extract at least five S.M.A.R.T. attributes into a dictionary.

On my machine, the drive was detected as: “*/dev/sda -d ata*”

So, I have to use */dev/sda* as the device identifier.

```
Lab2 python smart_monitor.py
Parsed S.M.A.R.T. attributes:
Raw_Read_Error_Rate: 0
Reallocate_NAND_Blk_Cnt: 0
Power_On_Hours: 2611
Power_Cycle_Count: 5924
Temperature_Celsius: 55
Offline_Uncorrectable: 0
UDMA_CRC_Error_Count: 0
BLAISE Lab2 ~main . ?3
```

Activity 3.2: Threshold-Based Health Checks

The objective was to extend the S.M.A.R.T. monitoring program by defining threshold rules for potentially problematic drive conditions and generating a warning when a threshold is exceeded.

For this activity, I used the following rules:

Attribute	Threshold	Action
Reallocate_NAND_Blk_Cnt	> 0	Back up important data
Offline_Uncorrectable	> 0	Back up data and investigate the drive
UDMA_CRC_Error_Count	> 0	Check the drive interface/connection
Temperature_Celsius	> 50°C	Improve cooling and airflow

The error-count thresholds treat any non-zero value as a warning condition. A temperature above 50°C was used as the warning threshold, following the example given in the lab. The thresholds are simple monitoring rules and are not universal guarantees of drive failure.

Programming

I extended the Activity 3.1 program with a *check_health()* function that compares the parsed S.M.A.R.T. values against the selected thresholds and reports a warning and recommended action when a threshold is exceeded.

The implementation is provided in the accompanying Python file.

Actual Result

The program produced the following S.M.A.R.T. values:

```
Lab2 python smart_monitor.py
Parsed S.M.A.R.T. attributes:
Raw_Read_Error_Rate: 0
Reallocate_NAND_Blks_Cnt: 0
Power_On_Hours: 2612
Power_Cycle_Count: 5924
Temperature_Celsius: 42
Offline_Uncorrectable: 0
UDMA_CRC_Error_Count: 0

Health Check:
No threshold warnings detected.
BLAISE Lab2 ^main . ?3
```

Small Discussion

None of the selected thresholds was exceeded during the measurement. The reallocation, uncorrectable-error, and CRC-error values were all zero, while the drive temperature was 42°C, which was below the 50°C warning threshold.

Therefore, the program correctly reported “*No threshold warnings detected.*”

This demonstrates how simple threshold rules can provide an automated first-level check of storage-device health. However, these rules should be interpreted as indicators rather than guarantees that a drive will not fail.

Activity 3.3: Trend Logging of S.M.A.R.T. Data

Now we are going to log the parsed S.M.A.R.T. attributes to a CSV or JSON file, including a timestamp, for at least three occasions. We will then examine which attributes changed and discuss whether those changes are meaningful or expected.

1. Programming

We extended the program from Activity 3.2 by adding a function that saves each S.M.A.R.T. reading to a CSV file.

```

Lab2 python smart_monitor.py
Current S.M.A.R.T. attributes:
Raw_Read_Error_Rate: 0
Reallocate_NAND_Blk_Cnt: 0
Power_On_Hours: 2612
Power_Cycle_Count: 5924
Temperature_Celsius: 42
Offline_Uncorrectable: 0
UDMA_CRC_Error_Count: 0
S.M.A.R.T. data logged at 2026-09-25 15:52:42
BLAISE Lab2 ^main 74

```

Actual Result

smart_log.csv

```

Timestamp,Raw_Read_Error_Rate,Reallocate_NAND_Blk_Cnt,Power_On_Hours,Power_Cycle_Count,Temperature_Celsius,Offline_U
ncorrectable,UDMA_CRC_Error_Count

2026-09-25 15:52:42,0,0,2612,5924,42,0,0
2026-09-25 15:59:02,0,0,2612,5924,41,0,0
2026-09-25 15:59:03,0,0,2612,5924,41,0,0
2026-09-25 15:59:04,0,0,2612,5924,42,0,0
2026-09-25 15:59:06,0,0,2612,5924,42,0,0
2026-09-25 15:59:12,0,0,2612,5924,42,0,0

```

The logged measurements showed that most S.M.A.R.T. attributes remained unchanged during the observation period.

The main change was in **temperature**, which varied between **41°C and 42°C**.

The following attributes remained constant across all six readings:

- *Raw_Read_Error_Rate = 0*
- *Reallocate_NAND_Blk_Cnt = 0*
- *Power_On_Hours = 2612*
- *Power_Cycle_Count = 5924*
- *Offline_Uncorrectable = 0*
- *UDMA_CRC_Error_Count = 0*

Therefore, no increase in the monitored error-related attributes was observed during the six measurements.

Trend Detection

To determine whether any S.M.A.R.T. attributes increased during the observation period, the log was read back and the first and last values were compared.

The following function was used:

```

import csv
def find_increasing(log_file="smart_log.csv"):
    with open(log_file, newline="") as file:
        readings = list(csv.DictReader(file))
    if len(readings) < 2:

```

```

    print("Need at least two logged readings to detect a trend.")
    return {}
first = readings[0]
last = readings[-1]
trends = {}
for key in first:
    if key == "Timestamp":
        continue
    first_value = float(first[key])
    last_value = float(last[key])
    if last_value > first_value:
        trends[key] = (first_value, last_value)
        print(
            f"{key} increased: "
            f"{first_value} -> {last_value}"
        )
if not trends:
    print("No S.M.A.R.T. attributes increased.")
return trends

```

Actual Trend Result

The first and last logged readings were:

First reading: 15:52:42
Last reading: 15:59:12

There was **no increase in any of the logged S.M.A.R.T. attributes** during this observation period.

The trend-detection function therefore produced: “***No S.M.A.R.T. attributes increased.***”

This agrees with the recorded data: the error-related attributes remained at zero, ***Power_On_Hours*** remained at 2612, and ***Power_Cycle_Count*** remained at 5924. Only temperature changed, fluctuating between 41°C and 42°C, but it returned to the same value as the first reading rather than increasing overall. Blaise_Izerimana_Lab2

Small Discussion

The trend results show why different S.M.A.R.T. attributes need to be interpreted differently. An increase in an error-related attribute such as ***Reallocate_NAND_Blks_Cnt*** could be a meaningful warning sign, whereas ***Power_On_Hours*** naturally increases as the drive is used and is not by itself evidence of failure.

In this short observation period, none of the error-related attributes increased. The small temperature variation from 42°C to 41°C and back to 42°C is better interpreted as normal short-term operating variation rather than degradation.

Activity 3.4: False Positives and False Negatives

1. False Positive

A **false positive** occurs when the monitoring system reports a problem even though there is no significant underlying problem.

For example, our temperature rule is: **Temperature > 50°C → warning**

Suppose the SSD temporarily reaches **51°C** while performing a demanding task. The program would generate a warning even if the temperature returns to normal shortly afterwards. This would be a false positive caused by a temporary condition.

2. False Negative

A **false negative** occurs when the monitoring system does not generate a warning even though a genuine problem may be developing.

For example, our rule: **Reallocate_NAND_Blks_Cnt > 0 → warning**

only reacts after the value becomes greater than zero. A drive could potentially show other changes or degradation before this particular value crosses the threshold. Therefore, relying on individual threshold values may fail to detect some early or complex failure patterns.

3. Proposed Improvement

One improvement would be to require a threshold violation to occur over **several consecutive readings** before generating a warning.

For example: **Temperature > 50°C for three consecutive readings → warning.**

This would reduce false positives caused by temporary temperature spikes.

It would also make the monitoring system consider the **trend and persistence** of a condition rather than reacting to a single measurement.

4. Machine Learning Approach

A machine-learning system could consider several S.M.A.R.T. attributes simultaneously instead of relying only on individual fixed thresholds.

For example, it could analyze: Temperature, Power-on hours, Reallocation counts, Uncorrectable errors, CRC errors, Changes in these values over time

With enough historical drive-health data, a model could learn combinations and trends associated with potential failures.

However, such an approach would require appropriate historical data for training and validation, whereas the threshold-based approach can be implemented immediately without a training dataset.

Conclusion

Activities 3.1–3.4 demonstrated a basic automated S.M.A.R.T. monitoring system. The system first collected and parsed S.M.A.R.T. attributes, then applied threshold-based health checks and logged measurements over

time. The six observations showed stable error-related attributes and only a small temperature fluctuation between **41°C and 42°C**.

The exercise also showed a limitation of simple threshold monitoring: a single measurement can sometimes produce a false warning or fail to capture a developing problem. Requiring repeated threshold violations and analyzing trends can improve the reliability of the monitoring system.

Reflections

Reflection 1: How Far Can You Trust a Benchmark?